# Adding Asynchronous Actions to Redux

When we last looked at our asynchronous actions before we added in Redux, what we basically had was a combination of our ActionCable calls and our reducer calls. The problem with this is that the combination is somewhat awkward and requires always remembering that the two parts need to be called together. What we really want is for our store, which is now provided by Redux, to allow us to mix these asynchronous actions with our regular reducer calls.

To do this, we are going to use an add-on to Redux called Redux Thunk.[75] Redux Thunk is actually based on a pretty small idea. Up until now, the actions we've been passing to our reducer have just been data objects. What Redux Thunk asks is, "What if those arguments are something else?"

Redux Thunk allows for a different kind of argument to be passed to `dispatch`, namely a function that itself returns a function. Hence the name Redux Thunk, because *thunk* is a generic term for a function that returns a function.

You dispatch the function by calling it, so where we before had calls like `dispatch({type: "doSomething"})`, we now will have `dispatch(doSomething())` where the call to `doSomething` itself returns a function.

I pause here to note that it is super common in the React world to wrap action calls to reducers inside functions that return the action object. So rather than `dispatch({type: "doSomethng"})`, we'd write `dispatch(doSomething())` where `doSomething` is defined as:

```
const doSomething = () => {
    return { type: "doSomething" }
}
```

The argument for doing this extra function definition is that the functions are easier to manage in complex cases. We haven't been doing that so far because it seems like a bit too much boilerplate code for our purposes, but it's very common in practice. (As Betsy Haibel, co-founder of the engineering education